

Lecture 3:

LDA and regularized regression analysis

Vegard Høghaug Larsen

GRA4160: Predictive modelling with machine learning

January 21st 2026

Progress on mini project

- ▶ Presentation: March 18th
- ▶ I've added an excel sheet to Itslearning. Please fill in your group members and the topic you want to work on.

- ▶ Presentation: March 18th
- ▶ I've added an excel sheet to Itslearning. Please fill in your group members and the topic you want to work on.

Some possible topics:

- ▶ **Image Classification:** Train a model or models to classify images into different categories, such as animals, objects, etc. using a public dataset like: CIFAR or MNIST.
- ▶ **Sentiment Analysis:** Train a classifier to predict the sentiment (positive, negative, neutral) of a piece of text (e.g., movie reviews, product reviews, etc.) using a public dataset like Large Movie Review Dataset or Yelp reviews.
- ▶ **Macroeconomic forecasting:** Predict macro variables with ML methods such as random forest or neural nets. One possible dataset is FRED-MD.

Plan for today:

1. Recap from last week: Supervised learning
2. Linear discriminant analysis
3. Regularized regression analysis
4. Exercise: Predicting house prices

- ▶ **Machine Learning** enables computers to learn patterns from data.
- ▶ Three main paradigms:
 - ▶ **Supervised** learning (labeled data).
 - ▶ **Unsupervised** learning (unlabeled data).
 - ▶ **Reinforcement** learning (reward-based).
- ▶ **Bias-Variance Trade-off**: Balancing underfitting (*high bias*) vs. overfitting (*high variance*).
- ▶ Ultimate goal: **Generalize well** to new, unseen data.

▶ **Linear Regression**

- ▶ Predicts continuous outcomes.
- ▶ Parametric model minimizing MSE.
- ▶ Useful for both prediction & inference.

▶ **k-Nearest Neighbors (kNN)**

- ▶ Non-parametric, instance-based.
- ▶ Classification/regression by majority vote/mean of nearest neighbors.
- ▶ Choosing k is crucial to avoid over/underfitting.

▶ **Naive Bayes**

- ▶ Relies on Bayes' Theorem with a “naive” independence assumption.
- ▶ Often used for text classification (e.g., spam detection).
- ▶ Simple, fast, surprisingly effective despite strong assumptions.

Introduction to Linear Discriminant Analysis (LDA)

- ▶ **Definition:** LDA is both a *classification* and *dimensionality reduction* technique.
- ▶ **Core idea:** Project data onto a lower-dimensional space (often 1D) that maximizes class separability.

Introduction to Linear Discriminant Analysis (LDA)

- ▶ **Definition:** LDA is both a *classification* and *dimensionality reduction* technique.
- ▶ **Core idea:** Project data onto a lower-dimensional space (often 1D) that maximizes class separability.
- ▶ **Typical Uses:**
 - ▶ Face recognition
 - ▶ Medical diagnosis
 - ▶ Market research

Decision Boundary / Split points into classes
Classification
3 Features into 2 Features / Distance metric
Vs. K-NN metric

Introduction to Linear Discriminant Analysis (LDA)

- ▶ **Definition:** LDA is both a *classification* and *dimensionality reduction* technique.
- ▶ **Core idea:** Project data onto a lower-dimensional space (often 1D) that maximizes class separability.
- ▶ **Typical Uses:**
 - ▶ Face recognition
 - ▶ Medical diagnosis
 - ▶ Market research
- ▶ **Why LDA?**
 - ▶ Simple, interpretable linear boundaries.
 - ▶ Often effective when class distributions are approximately Gaussian.
 - ▶ Can reduce noise by focusing on most discriminative directions.

When to Use LDA

- ▶ **Two or more classes:** LDA can handle multi-class classification by finding multiple discriminant components.
- ▶ **Data separability:** Works well when classes are well-separated and covariance is similar across classes.
- ▶ **Dimensionality reduction:** When you need a lower-dimensional representation for further analysis or visualization.

When to Use LDA

- ▶ **Two or more classes:** LDA can handle multi-class classification by finding multiple discriminant components.
- ▶ **Data separability:** Works well when classes are well-separated and covariance is similar across classes.
- ▶ **Dimensionality reduction:** When you need a lower-dimensional representation for further analysis or visualization.
- ▶ **Comparison to Other Methods:**
 - ▶ **Logistic Regression:** Also linear, but does not reduce dimensionality.
 - ▶ **Quadratic Discriminant Analysis (QDA):** Allows different covariance matrices per class, but is more complex.
 - ▶ **Nearest Neighbors:** Non-parametric, can be less interpretable in high dimensions.
 - ▶ **Principal Component Analysis (PCA):** Unsupervised, focuses on variance rather than class separation. *You can use PCA when you don't have classes or any labels in your data*

- ▶ **Objective:** Maximize between-class variance while minimizing within-class variance.
- ▶ **Scatter Matrices:**
 - ▶ $\mathbf{S}_B$ (Between-class scatter) captures how class means differ.
 - ▶ $\mathbf{S}_W$ (Within-class scatter) captures how data points vary around their class means.
- ▶ **Optimization:**

$$\mathbf{w} = \arg \max_{\mathbf{w}} \frac{\mathbf{w}^T \mathbf{S}_B \mathbf{w}}{\mathbf{w}^T \mathbf{S}_W \mathbf{w}}.$$

- ▶ The vector $\mathbf{w}$ defines the projection that best separates the classes in a low-dimensional subspace.

Deeper Dive: How $\mathbf{w}$ is Computed

- ▶ **Class Means:** Let $\mathbf{m}_i$ be the mean vector of class i , and $\mathbf{m}$ the overall mean of all classes combined.

$$\mathbf{m}_i = \frac{1}{N_i} \sum_{\mathbf{x} \in \text{class } i} \mathbf{x}, \quad \mathbf{m} = \frac{1}{N} \sum_{i=1}^C \sum_{\mathbf{x} \in \text{class } i} \mathbf{x}$$

- ▶ **Finding the Discriminant Directions:**

- ▶ We solve the generalized eigenvalue problem:

$$\mathbf{S}_W^{-1} \mathbf{S}_B \mathbf{w} = \lambda \mathbf{w}.$$

- ▶ The eigenvectors ($\mathbf{w}$) with the largest eigenvalues λ provide the **most discriminative** directions.

- ▶ **For Two Classes:**

- ▶ There is only one relevant discriminant direction.
- ▶ An analytic solution is given by:

$$\mathbf{w} \propto \mathbf{S}_W^{-1} (\mathbf{m}_1 - \mathbf{m}_2).$$

- ▶ **Result:** Project data onto these directions $\mathbf{w}$ to achieve maximum class separation in a lower-dimensional space.

Key Assumptions of LDA

1. **Normality:** Data in each class is drawn from a Gaussian (normal) distribution.
2. **Identical Covariance:** All classes share the same covariance matrix Σ .
3. **Statistical Independence of Features:** Usually assumed, though in practice mild correlations are tolerated.

Implications:

- ▶ If the covariance structures differ significantly between classes, *Quadratic Discriminant Analysis (QDA)* may be more appropriate.
- ▶ Violations of normality often do not drastically harm LDA, but may reduce its optimality.

Implementing LDA in Scikit-learn

▶ Key Class:

- ▶ `LinearDiscriminantAnalysis` from `sklearn.discriminant_analysis`.

▶ Common Parameters:

- ▶ `solver`: e.g. `svd`, `lsqr`, or `eigen`.
- ▶ `shrinkage`: can improve robustness when data is limited or covariance matrices are ill-conditioned.
- ▶ `n_components`: controls dimensionality reduction ($\leq$ number of classes - 1).

▶ Basic Usage:

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
lda = LinearDiscriminantAnalysis(solver='svd', n_components=1)
lda.fit(X_train, y_train)
predictions = lda.predict(X_test)
```

▶ Model Attributes:

- ▶ `lda.coef_`: Class-wise linear coefficients.
- ▶ `lda.explained_variance_ratio_`: Variance explained by each discriminant component.
- ▶ `lda.means_`: Class means.

▶ **Data Preprocessing:**

- ▶ Scale or standardize features for more stable covariance estimation.
- ▶ Verify no extreme multicollinearity issues.

▶ **Model Evaluation:**

- ▶ Use cross-validation to compare accuracy or other metrics.
- ▶ Check confusion matrices for class-level performance.

▶ **When LDA Shines:**

- ▶ Moderate to large datasets where class distributions are somewhat separable.
- ▶ Preprocessing step for dimensionality reduction (visualization or feeding to other models).

- ▶ **Key Takeaway:** LDA is a powerful, interpretable linear approach for classification and dimension reduction, especially under roughly Gaussian assumptions with shared covariance.

- ▶ **Goal:** Reduce model overfitting by penalizing large coefficients.
- ▶ **Idea:** Add a penalty term to the cost function $\sum(y - \hat{y})^2$.
- ▶ **Key benefits:**
 - ▶ Controls the magnitude of coefficients.
 - ▶ Can improve predictive performance on unseen data.
 - ▶ Some methods also perform *feature selection*.
- ▶ **We will cover:**
 1. Ridge Regression (L2 penalty)
 2. Lasso Regression (L1 penalty)
 3. Elastic Net (Combination of L1 and L2)

- ▶ **Key Idea:** Uses an ℓ_2 -norm penalty on coefficients.

$$J^{\text{ridge}}(\beta) = \frac{1}{2n} \sum_i \left(y_i - \beta_0 - \sum_j \beta_j x_{ij} \right)^2 + \lambda_r \sum_j \beta_j^2.$$

- ▶ λ_r (the regularization parameter) balances fitting error and coefficient shrinkage.
- ▶ **Effects:**
 - ▶ Coefficients are **shrunk** but *rarely* driven to zero exactly.
 - ▶ Useful if many features are expected to have small, nonzero effects.
- ▶ **Interpretation:**
 - ▶ Larger $\lambda_r \implies$ stronger shrinkage.
 - ▶ Often improves stability in cases of multicollinearity.

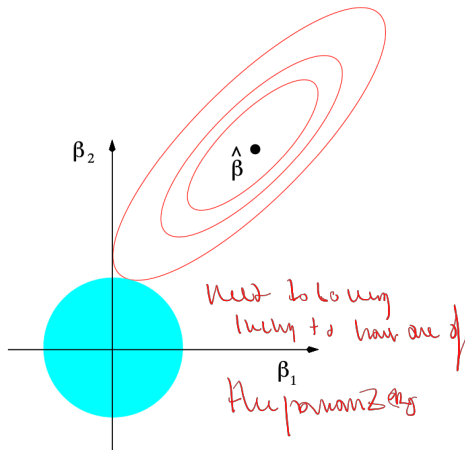
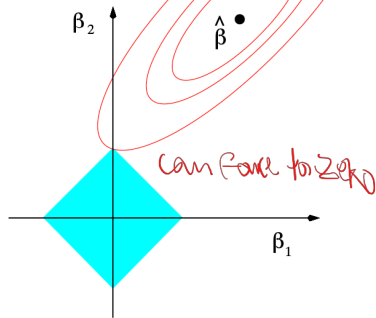
- ▶ **Key Idea:** Uses an ℓ_1 -norm penalty on coefficients.

$$J^{\text{lasso}}(\beta) = \frac{1}{2n} \sum_i \left(y_i - \beta_0 - \sum_j \beta_j x_{ij} \right)^2 + \lambda_\ell \sum_j |\beta_j|.$$

- ▶ λ_ℓ controls the amount of shrinkage and potential sparsity.
- ▶ **Main Feature:**
 - ▶ Coefficients can become exactly **zero**, performing **feature selection**.
 - ▶ **LARS algorithm** can efficiently compute the entire regularization path.
- ▶ **Common Use Cases:**
 - ▶ High-dimensional datasets where we suspect some features have no real effect.
 - ▶ Desire a simpler model with fewer active predictors.

Visualizing Lasso and Ridge

Indifference
lines



- ▶ **Ridge:** Penalty regions are circular (ℓ_2 -norm). Coefficients shrink uniformly.
- ▶ **Lasso:** Penalty regions are diamond-shaped (ℓ_1 -norm). Corners can force coefficients to zero.

- **Definition:** Combines both ℓ_1 and ℓ_2 penalties.

$$J^{\text{elastic net}}(\beta) = \frac{1}{2n} \sum_i (y_i - \beta_0 - \sum_j \beta_j x_{ij})^2 + \lambda_e \left((1 - \alpha) \sum_j \beta_j^2 + \alpha \sum_j |\beta_j| \right).$$

- $\alpha \in [0, 1]$ controls the **mix** between ℓ_2 -like and ℓ_1 -like behavior.
- **Why Elastic Net?**
 - Balances the strengths of ridge (stability) and lasso (sparsity).
 - Can outperform pure ridge/lasso in cases with correlated features.

L1 + L2 norm

IF variable selection ↑ Alpha

*outperforms
models that
use only ridge or lasso*

*Great for
hyperparameter tuning*

Comparing Ridge, Lasso, and Elastic Net

Ridge (ℓ_2):

- ▶ Coefficients get *shrunk* but *not* set to zero.
- ▶ Good when all features have small contributions.
- ▶ Helps reduce variance from correlated predictors.

Lasso (ℓ_1):

- ▶ Can set some coefficients to **zero** (*feature selection*).
- ▶ More useful when many features may be truly irrelevant.
- ▶ Can struggle when predictors are highly correlated (tends to pick one).

Elastic Net:

- ▶ A convex combination of ℓ_1 and ℓ_2 penalties.
- ▶ Offers some sparsity plus stability.
- ▶ Tuning involves λ *and* α .

Key Takeaway:

- ▶ Choose based on data structure and desired features.
- ▶ Use cross-validation to find optimal regularization parameters.

Choosing the Regularization Parameter λ

- ▶ λ (or λ_r , λ_ℓ , etc.) controls the penalty strength:
 - ▶ $\lambda = 0 \rightarrow$ reverts to ordinary least squares (no penalty).
 - ▶ $\lambda \rightarrow \infty \rightarrow$ shrinks all coefficients toward 0.
- ▶ **Cross-Validation:**
 - ▶ Systematically vary λ over a grid of values.
 - ▶ Evaluate prediction error (MSE, MAE, etc.) on the validation set.
 - ▶ Pick λ that minimizes validation error or has the best trade-off (e.g., 1-SE rule).
- ▶ **Practical Tips:**
 - ▶ Standardize features before applying regularization to ensure fair coefficient shrinkage.
 - ▶ Watch for high correlation among features (especially for Lasso).

► Scikit-learn Implementations:

- `Ridge()` for ridge regression.
- `Lasso()` for lasso regression.
- `ElasticNet()` for elastic net.

► Example (Ridge):

```
from sklearn.linear_model import Ridge
ridge_reg = Ridge(alpha=1.0)
ridge_reg.fit(X_train, y_train)
y_pred = ridge_reg.predict(X_test)
```

► Hyperparameter Tuning:

- Use `GridSearchCV` or `RandomizedSearchCV` to find the best α .
- For `ElasticNet`, tune both α (mixing ratio) and λ .

► Common Pitfalls:

- Forgetting to scale data can bias penalty terms.
- Over-penalizing: too high λ can underfit.